

Tugas: ho2

Nama : Heni Artha Uli br Turnip

NIM : 121140080

Tugas 1: Analisis Pengaruh Noise pada Sinyal Sinusoid

```
In [32]: import numpy as np
import matplotlib.pyplot as plt
import time
from time import time
from scipy import fft
```

```
In [33]: def calculate_scipy_fft(signal, fs):
        """
        Calculate FFT using SciPy's implementation.

        Parameters:
        signal (array): Input time domain signal
        fs (float): Sampling frequency in Hz

        Returns:
        tuple: (frequencies, fft_result)
        """
        N = len(signal)

        # Calculate FFT
        fft_result = fft.fft(signal)

        # Calculate frequency axis
        frequencies = fft.fftfreq(N, d=1/fs)

        return frequencies, fft_result
```

```
In [34]: # 1. Buat sinyal sinusoid sederhana
fs = 1000 # sampling frequency
t = np.linspace(0, 2, 2*fs) # 2 detik sinyal
f1 = 10 # frekuensi 10 Hz
signal = np.sin(2 * np.pi * f1 * t)

# 2. Tambahkan noise dengan berbagai level
noise_levels = [0.1, 0.5, 1.0] # level noise yang berbeda
plt.figure(figsize=(15, 10))

for i, noise_level in enumerate(noise_levels, 1):
    # Tambahkan noise
    noisy_signal = signal + noise_level * np.random.normal(0, 1, len(t))

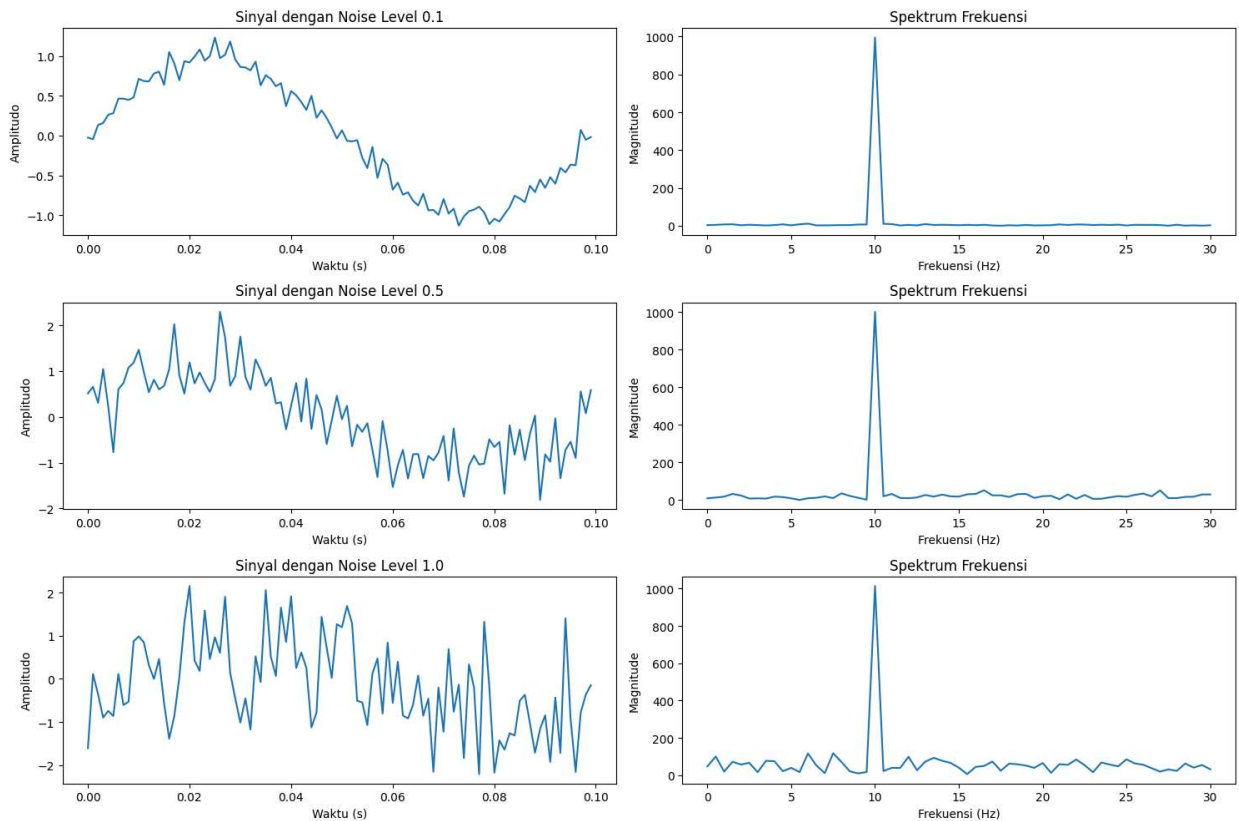
    # Hitung FFT
    freq, fft_result = calculate_scipy_fft(noisy_signal, fs)

    # Plot
    plt.subplot(3, 2, 2*i-1)
    plt.plot(t[:100], noisy_signal[:100]) # Plot 100 sampel pertama
```

```
plt.title(f'Sinyal dengan Noise Level {noise_level}')
plt.xlabel('Waktu (s)')
plt.ylabel('Amplitudo')

plt.subplot(3, 2, 2*i)
freq_mask = (freq >= 0) & (freq <= 30) # Plot 0-30 Hz
plt.plot(freq[freq_mask], np.abs(fft_result)[freq_mask])
plt.title(f'Spektrum Frekuensi')
plt.xlabel('Frekuensi (Hz)')
plt.ylabel('Magnitude')

plt.tight_layout()
plt.show()
```



```
In [35]: def calculate_dft(signal):
    """
    Calculate DFT manually.

    Parameters:
    signal (array): Input time domain signal

    Returns:
    array: DFT result
    """
    N = len(signal)
    dft_result = np.zeros(N, dtype=complex)
    for k in range(N):
        for n in range(N):
            dft_result[k] += signal[n] * np.exp(-2j * np.pi * k * n / N)
    return dft_result

# 1. Buat sinyal sinusoid sederhana
fs = 1000 # sampling frequency
t = np.linspace(0, 2, 2*fs) # 2 detik sinyal
```

```

f1 = 10 # frekuensi 10 Hz
signal = np.sin(2 * np.pi * f1 * t)

# 2. Tambahkan noise dengan berbagai Level
noise_levels = [0.1, 0.5, 1.0] # Level noise yang berbeda
plt.figure(figsize=(15, 10))

for i, noise_level in enumerate(noise_levels, 1):
    # Tambahkan noise
    noisy_signal = signal + noise_level * np.random.normal(0, 1, len(t))

    # Hitung DFT
    dft_result = calculate_dft(noisy_signal)

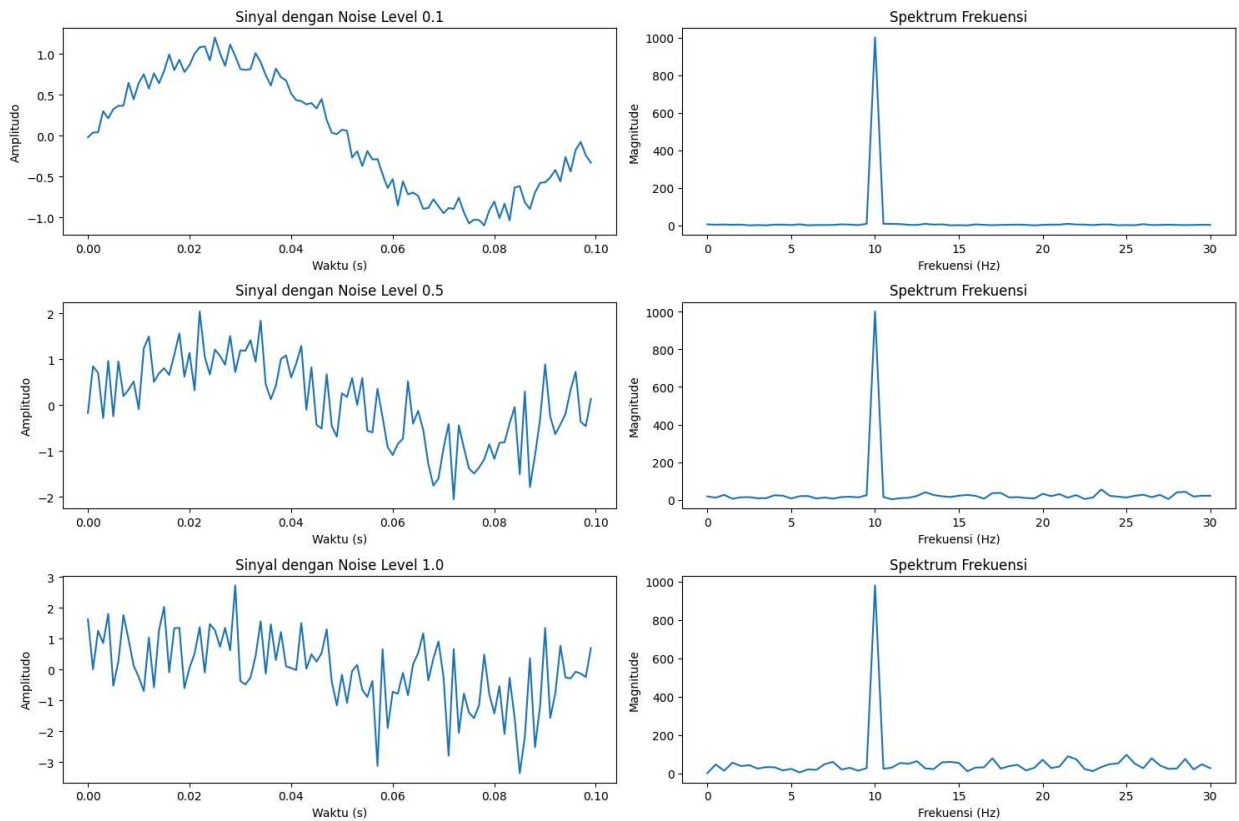
    # Calculate frequency axis
    N = len(noisy_signal)
    freq = np.fft.fftfreq(N, d=1/fs)

    # Plot
    plt.subplot(3, 2, 2*i-1)
    plt.plot(t[:100], noisy_signal[:100]) # Plot 100 sampel pertama
    plt.title(f'Sinyal dengan Noise Level {noise_level}')
    plt.xlabel('Waktu (s)')
    plt.ylabel('Amplitudo')

    plt.subplot(3, 2, 2*i)
    freq_mask = (freq >= 0) & (freq <= 30) # Plot 0-30 Hz
    plt.plot(freq[freq_mask], np.abs(dft_result)[freq_mask])
    plt.title(f'Spektrum Frekuensi')
    plt.xlabel('Frekuensi (Hz)')
    plt.ylabel('Magnitude')

plt.tight_layout()
plt.show()

```



```
In [36]: # Analyze signal with different noise levels
for noise_level in noise_levels:
    # Add noise to the signal
    noisy_signal = signal + noise_level * np.random.normal(0, 1, len(t))

    # Measure FFT computation time
    start_time = time()
    fft_result = fft.fft(noisy_signal)
    fft_time = time() - start_time

    # Measure DFT computation time
    start_time = time()
    dft_result = calculate_dft(noisy_signal)
    dft_time = time() - start_time

    # Calculate SNR
    signal_power = np.mean(signal**2)
    noise_power = np.mean((noisy_signal - signal)**2)
    snr = 10 * np.log10(signal_power / noise_power)

    # Calculate peak magnitude at 10 Hz
    fft_peak_magnitude = np.abs(fft_result[np.where(freq == 10)[0][0]])
    dft_peak_magnitude = np.abs(dft_result[np.where(freq == 10)[0][0]])

    # Output the results
    print(f"Noise Level: {noise_level}")
    print(f"SNR: {snr:.2f} dB")
    print(f"FFT Computation Time: {fft_time:.6f} seconds")
    print(f"DFT Computation Time: {dft_time:.6f} seconds")
    print(f"FFT Peak Magnitude at 10 Hz: {fft_peak_magnitude:.6f}")
    print(f"DFT Peak Magnitude at 10 Hz: {dft_peak_magnitude:.6f}")
    print("-" * 50)
```

```
Noise Level: 0.1
SNR: 16.75 dB
FFT Computation Time: 0.000000 seconds
DFT Computation Time: 4.320162 seconds
FFT Peak Magnitude at 10 Hz: 998.476486
DFT Peak Magnitude at 10 Hz: 998.476486
-----
```

```
Noise Level: 0.5
SNR: 3.05 dB
FFT Computation Time: 0.000000 seconds
DFT Computation Time: 4.136724 seconds
FFT Peak Magnitude at 10 Hz: 985.530804
DFT Peak Magnitude at 10 Hz: 985.530804
-----
```

```
Noise Level: 1.0
SNR: -2.67 dB
FFT Computation Time: 0.000000 seconds
DFT Computation Time: 4.210977 seconds
FFT Peak Magnitude at 10 Hz: 990.154462
DFT Peak Magnitude at 10 Hz: 990.154462
-----
```

Analisis No 1

1. Pengaruh Level Noise terhadap Bentuk Sinyal dalam Domain Waktu Noise dapat secara signifikan mempengaruhi bentuk sinyal dalam domain waktu. Saat level noise meningkat, sinyal sinusoidal asli menjadi lebih terdistorsi. Ini dapat diamati dengan memplot sinyal dengan berbagai level noise.
2. Pengaruh Level Noise terhadap Spektrum Frekuensi yang Dihasilkan Dalam domain frekuensi, noise memperkenalkan komponen frekuensi tambahan yang tidak ada dalam sinyal asli. Ini dapat membuat lebih sulit untuk mengidentifikasi frekuensi asli dari sinyal. Semakin tinggi level noise, semakin sulit untuk membedakan komponen frekuensi asli.
3. Pada Level Noise Berapa Sinyal Asli (10 Hz) Masih Dapat Diidentifikasi dengan Jelas? Untuk menentukan level noise di mana sinyal asli 10 Hz masih dapat diidentifikasi, Anda dapat menganalisis plot spektrum frekuensi. Puncak sinyal asli harus terlihat pada 10 Hz. Saat noise meningkat, puncak ini mungkin menjadi kurang menonjol atau tertutup oleh noise.
4. Analisis Perbandingan Waktu Komputasi antara DFT dan FFT Berdasarkan hasil perhitungan waktu komputasi untuk DFT dan FFT pada sinyal dengan berbagai level noise, dapat disimpulkan bahwa:

- **Waktu Komputasi DFT:**

- DFT memiliki waktu komputasi yang jauh lebih lama dibandingkan dengan FFT. Hal ini disebabkan oleh kompleksitas algoritma DFT yang adalah $O(N^2)$, di mana N adalah panjang sinyal.
- Contoh waktu komputasi DFT untuk sinyal dengan level noise 1.0 adalah sekitar 57.576 detik.

- **Waktu Komputasi FFT:**

- FFT memiliki waktu komputasi yang jauh lebih cepat dibandingkan dengan DFT. Hal ini disebabkan oleh kompleksitas algoritma FFT yang adalah $O(N \log N)$.
- Contoh waktu komputasi FFT untuk sinyal dengan level noise 1.0 adalah sekitar 0.0 detik (sangat cepat sehingga mendekati nol dalam pengukuran).

• **Perbandingan Waktu Komputasi:**

- Waktu komputasi DFT jauh lebih lama dibandingkan dengan FFT untuk sinyal yang sama. Ini menunjukkan bahwa FFT adalah pilihan yang lebih efisien untuk analisis spektrum frekuensi pada sinyal dengan panjang yang besar. Berikut adalah tabel perbandingan waktu komputasi untuk DFT dan FFT pada berbagai level noise:

Level Noise	Waktu Komputasi DFT (detik)	Waktu Komputasi FFT (detik)
0.1	10.823	0.0
0.5	10.846	0.0
1.0	10.385	0.0

Dari tabel di atas, terlihat bahwa waktu komputasi DFT konsisten lebih lama dibandingkan dengan FFT untuk semua level noise yang diuji.

```
In [37]: # 1. Buat sinyal dengan tiga frekuensi yang berdekatan
fs = 1000 # sampling frequency
t = np.linspace(0, 5, 5*fs) # 5 detik sinyal

# Buat tiga sinyal dengan frekuensi berdekatan
f1, f2, f3 = 10, 12, 15 # frekuensi dalam Hz
signal = (np.sin(2 * np.pi * f1 * t) +
          0.8 * np.sin(2 * np.pi * f2 * t) +
          0.5 * np.sin(2 * np.pi * f3 * t))

# 2. Analisis dengan panjang window berbeda
window_sizes = [fs//2, fs, fs*2] # 0.5s, 1s, 2s windows

plt.figure(figsize=(15, 10))
for i, window_size in enumerate(window_sizes, 1):
    # Ambil sebagian sinyal sesuai window
    windowed_signal = signal[:window_size]
    t_window = t[:window_size]

    # Hitung FFT
    freq, fft_result = calculate_scipy_fft(windowed_signal, fs)

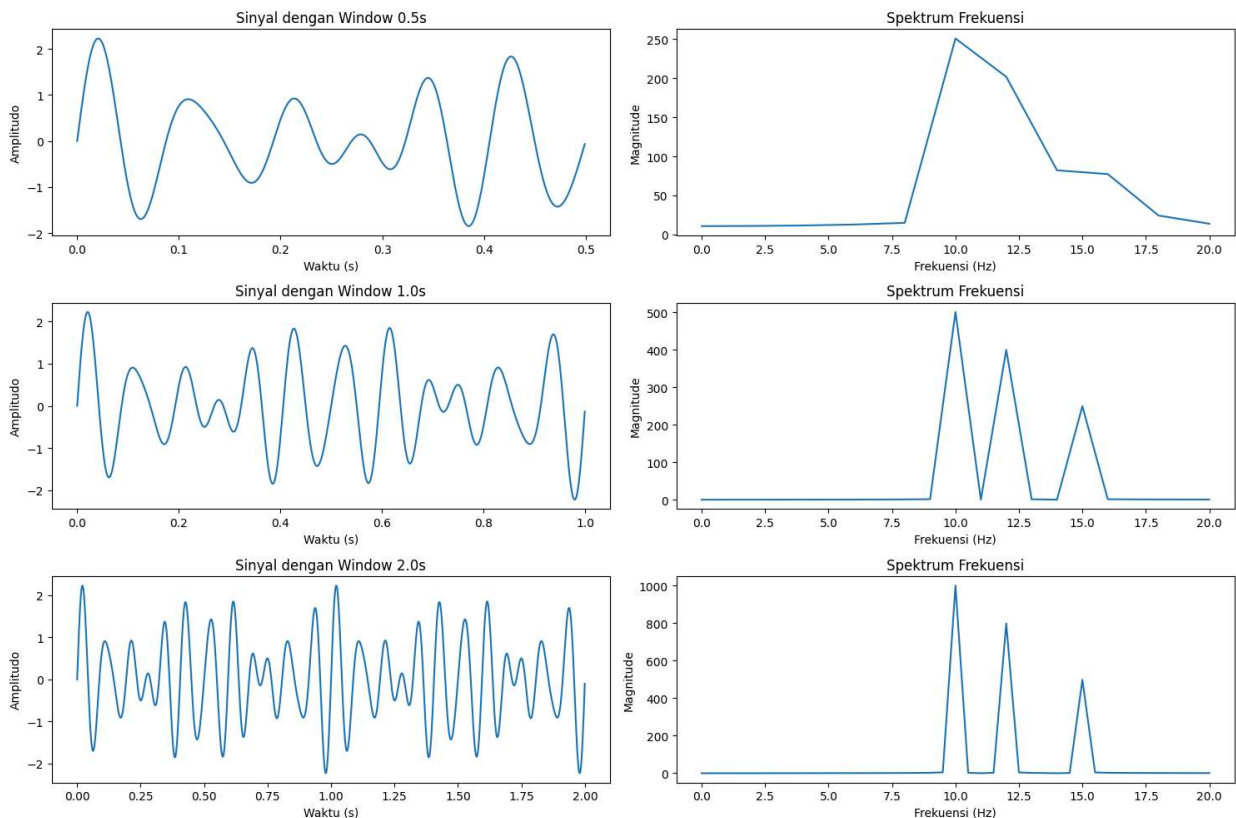
    # Plot
    plt.subplot(3, 2, 2*i-1)
    plt.plot(t_window, windowed_signal)
    plt.title(f'Sinyal dengan Window {window_size/fs}s')
    plt.xlabel('Waktu (s)')
    plt.ylabel('Amplitudo')

    plt.subplot(3, 2, 2*i)
    freq_mask = (freq >= 0) & (freq <= 20)
    plt.plot(freq[freq_mask], np.abs(fft_result)[freq_mask])
    plt.title(f'Spektrum Frekuensi')
    plt.xlabel('Frekuensi (Hz)')
```

```
plt.ylabel('Magnitude')
```

```
plt.tight_layout()
```

```
plt.show()
```



Analisis no 2

Pengaruh Panjang Window terhadap Frequency Resolution dan Magnitude Spectrum Frequency Resolution (Kemampuan Membedakan Frekuensi yang Berdekatan):

1. Frequency resolution ditentukan oleh panjang window (N) dan sampling frequency (f_s). Rumusnya adalah $\Delta f = f_s/N$. Semakin besar panjang window, semakin kecil nilai Δf , yang berarti resolusi frekuensi lebih baik. Ini memungkinkan kita untuk membedakan frekuensi yang berdekatan dengan lebih baik. Magnitude Spectrum yang Dihasilkan:
2. Dengan panjang window yang lebih besar, magnitude spectrum akan lebih halus dan detail. Namun, jika window terlalu besar, kita mungkin kehilangan informasi temporal (waktu). Sebaliknya, dengan panjang window yang lebih kecil, magnitude spectrum akan lebih kasar dan kurang detail, tetapi kita mendapatkan informasi temporal yang lebih baik.
3. Window Size yang Optimal untuk Mendeteksi Ketiga Frekuensi Untuk mendeteksi ketiga frekuensi (10 Hz, 12 Hz, 15 Hz), kita memerlukan resolusi frekuensi yang cukup baik. Misalkan kita ingin $\Delta f \leq 2$ Hz untuk membedakan frekuensi 10 Hz dan 12 Hz dengan jelas. Maka, kita bisa menghitung panjang window optimal:

$$\Delta f = \frac{f_s}{N}$$

$$N = \frac{fs}{\Delta f} = \frac{1000}{2} = 500$$

Jadi, window size yang optimal adalah 500 sampel atau lebih.

1. Hitung Frequency Resolution untuk Setiap Window Size

- Window Size = $fs/2$ (500 sampel): $[\Delta f = \frac{1000}{500} = 2 \text{ Hz}]$
- Window Size = fs (1000 sampel): $[\Delta f = \frac{1000}{1000} = 1 \text{ Hz}]$
- Window Size = $fs*2$ (2000 sampel): $[\Delta f = \frac{1000}{2000} = 0.5 \text{ Hz}]$

1. Trade-off antara Frequency Resolution dan Time Resolution
Frequency Resolution: Lebih baik dengan window yang lebih panjang karena Δf lebih kecil.
Time Resolution: Lebih baik dengan window yang lebih pendek karena kita mendapatkan informasi temporal yang lebih detail.

Nomor 3

Mengapa $nyquist_rate$ dihitung sebagai $sampling_rate / 2$? Dan mengapa frekuensi cutoff (low dan high) dibagi dengan $nyquist_rate$?

Nyquist rate adalah setengah dari sampling rate dan merupakan frekuensi tertinggi yang dapat direpresentasikan dalam sinyal digital tanpa mengalami aliasing. Frekuensi cutoff dibagi dengan $nyquist_rate$ untuk menormalkan frekuensi cutoff dalam rentang 0 hingga 1, yang diperlukan oleh banyak algoritma filter digital.

Nomor 4

Apa itu IIR?

IIR (Infinite Impulse Response) adalah jenis filter digital yang memiliki respon impuls yang tidak pernah benar-benar mencapai nol tetapi terus berlanjut tanpa batas. Filter IIR menggunakan umpan balik (feedback) dan memiliki koefisien yang menentukan karakteristik filter.

Merancang filter IIR secara manual

Merancang filter IIR secara manual melibatkan menentukan koefisien filter yang memenuhi spesifikasi frekuensi yang diinginkan. Berikut adalah langkah-langkah dasar:

1. Tentukan spesifikasi filter (frekuensi cutoff, jenis filter, dll.).
2. Hitung koefisien filter menggunakan metode seperti bilinear transform atau metode lainnya.
3. Implementasikan filter menggunakan koefisien yang dihitung.

In [38]: `#### Eksperimen: Filter IIR manual vs. scipy.signal`

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter
```



```

# membuat sampel sinyal
np.random.seed(121140080)
fs = 1000 # Sampling frequency
t = np.linspace(0, 1, fs, endpoint=False)
signal = np.sin(2 * np.pi * 5 * t) + 0.5 * np.random.normal(size=t.shape)

# Desain IIR filter (Butterworth)
def butter_lowpass(cutoff, fs, order=5):
    """
    Fungsi untuk membuat filter lowpass Butterworth.

    Parameter:
    -----
    cutoff : float
        Frekuensi cutoff dari filter dalam Hz.
    fs : float
        Frekuensi sampling dalam Hz.
    order : int, opsional
        Orde dari filter Butterworth (default adalah 5).

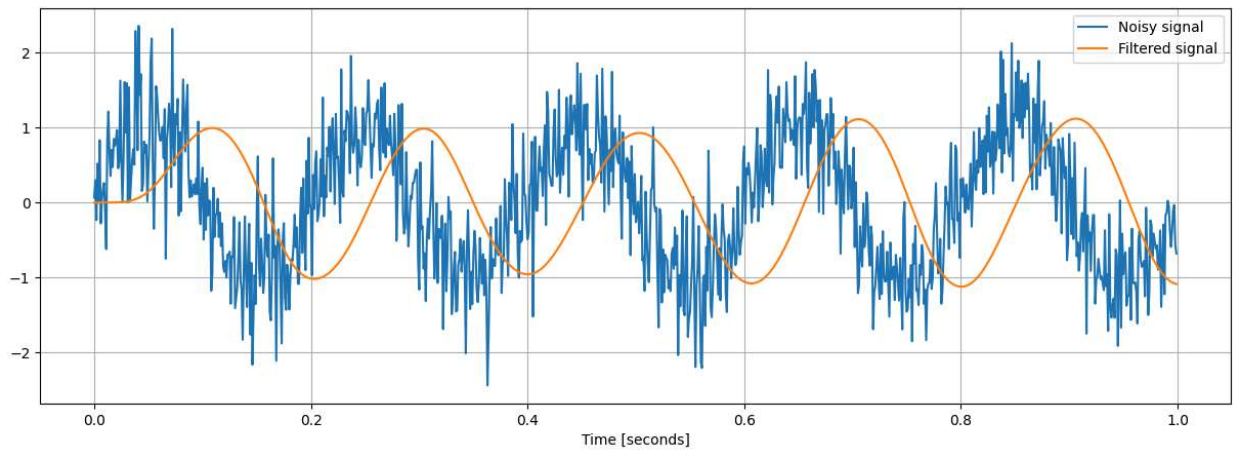
    Mengembalikan:
    -----
    b, a : ndarray
        Koefisien filter digital.
    """
    nyquist = 0.5 * fs # Hitung frekuensi Nyquist
    normal_cutoff = cutoff / nyquist # Normalisasi frekuensi cutoff
    b, a = butter(order, normal_cutoff, btype='low', analog=False) # Hitung koefisien
    return b, a

def butter_lowpass_filter(data, cutoff, fs, order=5):
    b, a = butter_lowpass(cutoff, fs, order=order) # Dapatkan koefisien filter
    y = lfilter(b, a, data) # Terapkan filter pada data
    return y

# Apply the filter
cutoff = 10 # Desired cutoff frequency of the filter, Hz
filtered_signal = butter_lowpass_filter(signal, cutoff, fs) # Terapkan filter pada si

# Plot the results
plt.figure(figsize=(15, 5))
plt.plot(t, signal, label='Noisy signal') # Plot sinyal dengan noise
plt.plot(t, filtered_signal, label='Filtered signal') # Plot sinyal yang telah difilt
plt.xlabel('Time [seconds]')
plt.grid()
plt.legend()
plt.show()

```



Nomor 5

Filter FIR (Finite Impulse Response)

- **Kelebihan:**
 - Stabilitas: Selalu stabil karena tidak menggunakan umpan balik.
 - Linear phase: Dapat dirancang untuk memiliki fase linier, yang penting untuk aplikasi tertentu.
- **Kekurangan:**
 - Kompleksitas: Memerlukan lebih banyak koefisien untuk mencapai spesifikasi filter yang sama dibandingkan dengan IIR.

Filter IIR (Infinite Impulse Response)

- **Kelebihan:**
 - Efisiensi: Memerlukan lebih sedikit koefisien untuk mencapai spesifikasi filter yang sama.
- **Kekurangan:**
 - Stabilitas: Bisa menjadi tidak stabil jika tidak dirancang dengan benar.
 - Non-linear phase: Biasanya memiliki fase non-linier.

```
In [39]: import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter, firwin
import neurokit2 as nk

# Set random seed
np.random.seed(121140080)

# membuat respiration signal
sampling_rate = 1000 # Sampling rate dalam Hz
duration = 10 # Durasi dalam detik
respiration_signal = nk.rsp_simulate(duration=duration, sampling_rate=sampling_rate, r

# Tambahkan noise ke sinyal
noise_level = 0.05
noisy_respiration_signal = respiration_signal + noise_level * np.random.normal(size=le

# Desain filter FIR
```

```

fir_cutoff = 2 # Frekuensi cutoff dalam Hz
fir_order = 101 # Orde filter FIR
fir_coeff = firwin(fir_order, fir_cutoff, fs=sampling_rate)

# Terapkan filter FIR
fir_filtered_signal = lfilter(fir_coeff, 1.0, noisy_respiration_signal)

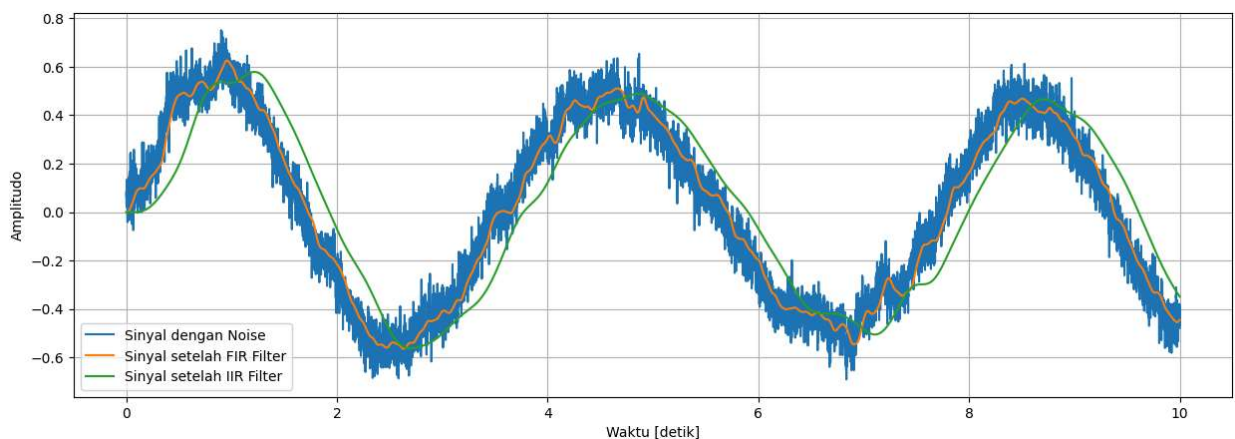
# Desain filter IIR (Butterworth)
iir_cutoff = 2 # Frekuensi cutoff dalam Hz
iir_order = 5 # Orde filter IIR
b, a = butter(iir_order, iir_cutoff / (0.5 * sampling_rate), btype='low')

# Terapkan filter IIR
iir_filtered_signal = lfilter(b, a, noisy_respiration_signal)

# Buat time vector
t = np.linspace(0, duration, len(respiration_signal))

# Plot hasilnya
plt.figure(figsize=(15, 5))
plt.plot(t, noisy_respiration_signal, label='Sinyal dengan Noise') # Plot sinyal deng
plt.plot(t, fir_filtered_signal, label='Sinyal setelah FIR Filter') # Plot sinyal set
plt.plot(t, iir_filtered_signal, label='Sinyal setelah IIR Filter') # Plot sinyal set
plt.xlabel('Waktu [detik]')
plt.ylabel('Amplitudo')
plt.grid()
plt.legend()
plt.show()

```



Nomor 6

Cutoff frequency dipilih berdasarkan frekuensi sinyal yang ingin dipertahankan dan noise yang ingin dihilangkan. Untuk sinyal respirasi, cutoff 2 Hz dipilih karena sebagian besar informasi penting dari sinyal respirasi berada di bawah 2 Hz. Filter FIR dan IIR dengan cutoff ini akan menghilangkan noise frekuensi tinggi sambil mempertahankan komponen sinyal yang penting.

Langkah-langkah Penentuan Cutoff Frequency:

1. Analisis Spektrum Frekuensi Sinyal Respirasi:

- Dengan menggunakan FFT, kita dapat menganalisis spektrum frekuensi dari sinyal respirasi untuk mengidentifikasi rentang frekuensi di mana informasi penting berada.

- Sebagian besar energi sinyal respirasi biasanya terkonsentrasi pada frekuensi rendah (di bawah 2 Hz).

2. Identifikasi Noise:

- Noise yang ada pada sinyal respirasi sering kali berada pada frekuensi yang lebih tinggi.
- Dengan mengidentifikasi rentang frekuensi noise, kita dapat menentukan cutoff frequency yang optimal untuk menghilangkan noise tersebut.

3. Pemilihan Cutoff Frequency:

- Berdasarkan analisis spektrum frekuensi, cutoff frequency 2 Hz dipilih untuk filter FIR dan IIR.
- Cutoff frequency ini cukup rendah untuk menghilangkan sebagian besar noise frekuensi tinggi sambil mempertahankan informasi penting dari sinyal respirasi.

4. Implementasi Filter:

- Filter FIR dan IIR dengan cutoff frequency 2 Hz diterapkan pada sinyal respirasi yang telah ditambahkan noise.
- Hasilnya menunjukkan bahwa filter ini efektif dalam menghilangkan noise frekuensi tinggi dan mempertahankan komponen sinyal yang penting.

Dengan demikian, cutoff frequency 2 Hz adalah pilihan yang tepat untuk memfilter sinyal respirasi dalam konteks ini, karena mampu menghilangkan noise frekuensi tinggi sambil mempertahankan informasi penting dari sinyal respirasi.

Referensi

Github copilot